

1 Intro and Softmax

- Trad. ML:** Humans design features, simple model designed on top. Pipeline: Feature extraction → Feature vector → simple classifier → prediction. We aren't good at designing features
- Feature:** numeric description of input.
- Deep learning:** learns features for us, using big (can capture enough) differentiable models.
- Depth:** Layer count

DL didn't blow up in the past due to limited compute, insufficient labeled data, and training challenges (optimizing non-convex function).

DL was made possible by regularization, better architectures introduced helpful inductive biases, advanced optimization beyond GD, clever initializations, pre-training, hardware advances, and large datasets.

1.1 Softmax Objective

The pre-activation is: $\tilde{o} = \mathbf{W}\tilde{\mathbf{x}} + \tilde{\mathbf{b}}$, where $\mathbf{W} \in \mathbb{R}^{K \times D}$, $\tilde{\mathbf{x}} \in \mathbb{R}^{D \times 1}$, $\tilde{\mathbf{b}} \in \mathbb{R}^K$, $\tilde{o} \in \mathbb{R}^K$.

- Maximize likelihood of true labels over training set: $\prod_{i=1}^N \Pr(y^{(i)} | \tilde{\mathbf{x}}^{(i)})$

- With one-hot labels: $\Pr(y^{(i)} | \tilde{\mathbf{x}}^{(i)}) = \prod_{j=1}^K (\tilde{y}_j^{(i)})^{y_j^{(i)}}$

- Plug into objective: $\prod_{i=1}^N \left(\prod_{j=1}^K (\tilde{y}_j^{(i)})^{y_j^{(i)}} \right)$

- Minimize the negative log-likelihood:

$$- \sum_{i=1}^N \left(\sum_{j=1}^K y_j^{(i)} \log(\tilde{y}_j^{(i)}) \right)$$

$$L_i(\tilde{\mathbf{y}}, \tilde{\mathbf{y}}) = - \sum_{j=1}^K y_j \log(\tilde{y}_j) = - \sum_j y_j \log \frac{\exp(o_j)}{\sum_k \exp(o_k)}$$

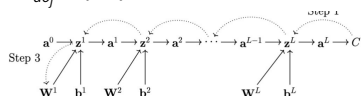
1.2 Softmax Gradient

Softmax guarantees non-negative probabilities, each

summing up to 1. Objective fn: $\prod_{i=1}^N \left(\prod_{j=1}^K (\tilde{y}_j^{(i)})^{y_j^{(i)}} \right)$.

For the gradient:

- $L_i(\mathbf{y}, \tilde{\mathbf{y}}) = - \sum_{j=1}^K y_j \log(\tilde{y}_j) = - \sum_j y_j \log \frac{\exp(o_j)}{\sum_k \exp(o_k)}$
- $= \sum_{j=1}^K y_j \log \left(\sum_k \exp(o_k) \right) - \sum_{j=1}^K y_j o_j$
- $= \log \left(\sum_k \exp(o_k) \right) - \sum_j y_j o_j$ (this is the cost function, and $\mathbf{y}$ is 1-hot).
- $\frac{dL_i}{do_j} = \frac{\partial}{\partial o_j} \left(\log \left(\sum_k \exp(o_k) \right) - \sum_j y_j o_j \right)$
- $\frac{\partial}{\partial o_j} \log \left(\sum_k \exp(o_k) \right) = \frac{\exp(o_j)}{\sum_k \exp(o_k)} = \tilde{y}_j$
- $\frac{\partial}{\partial o_j} \left(- \sum_j y_j o_j \right) = -y_j$
- $\frac{dL_i}{do_j} = \tilde{y}_j - y_j$



1.3 Universal Approximation Theorem

∀ continuous function f and desired accuracy $\varepsilon > 0$, ∃ MLP w/ one hidden layer that approximates f with error $< \varepsilon$.

Proof. A sigmoid neuron with large weights can approximate a step function. Two neurons with opposite weights form a localized bump function. Every continuous function is a sequence of narrow bumps.

- A sufficiently wide network can easily overfit the training data.
- Achieving good approximation may require very wide networks.
- The theorem says existence, not how to find the parameters.
- Perfect training fit does not imply good generalization.
- $\sigma(x) = \frac{1}{1+e^{-x}}$, simplest bump: $\sigma(x) - \sigma(x-1)$

1.4 Backprop Costs

The weight count in the layer is the no. of elements in the matrix. For instance, a 3×3 matrix has 9 weights. **Matrix multiplication dominates the cost of backpropagation.**

Forward pass has these sequence of operations:

$\mathbf{z}^m = \mathbf{W}^m \mathbf{a}^{m-1} + \mathbf{b}^m$, $\mathbf{a}^m = \sigma^m(\mathbf{z}^m)$ per layer, and for the output, $\mathbf{C} = \mathbf{C}(\mathbf{a}^L, \mathbf{y})$. Backward pass requires this

calculation for the output: $\nabla_{\mathbf{z}^L} \mathbf{C} = \nabla_{\mathbf{a}^L} \mathbf{C} \odot \sigma'^L(\mathbf{z}^L)$ and

for each layer, $\nabla_{\mathbf{z}^m} \mathbf{C} = ((\mathbf{W}^{m+1})^T \nabla_{\mathbf{z}^{m+1}} \mathbf{C}) \odot \sigma^{m'}(\mathbf{z}^m)$ and

$\nabla_{\mathbf{W}^m} \mathbf{C} = \nabla_{\mathbf{z}^m} \mathbf{C} (\mathbf{a}^{m-1})^T$. Which means:

- Forward pass has 1 matrix multiplication per layer, cost being no. of weights
- Backward pass requires 2 matrix multiplications per layer, cost is $2 \times$ no. of weights

- Total cost is $3 \times$ no. of weights

1.5 Overfitting and Regularization

Even linear models can overfit! If we achieve zero training loss, that is probably the result of **one feature separating the training data perfectly**, while the others are ignored by the model. Hence, the model assigns a very large weight to that feature and assigns near-zero weights to the other, making this no better than single-variable linear regression. With many features and limited training data, a feature will be perfectly predictive by chance.

2 Parameter Initialization

Initializing all weights (and as consequence, biases) to 0 results in all activations being 0, all weight gradients being 0, so no learning is done.

Initializing all weights to be the same, non-zero value results in identical activations, resulting in all weights in a layer receiving the same gradient, so all units in a layer remain identical throughout training.

2.1 Dropout, Choosing Parameters

Proof of $E[X] = 0 \wedge E[Y] = 0 \text{ idp.} \Rightarrow V[XY] = V[X]V[Y]$:

$$V[XY] = E[(XY)^2] - (E[XY])^2 = E[X^2 Y^2] - \underbrace{(E[X]E[Y])^2}_{=0}$$

$$= E[X^2 Y^2] = E[X^2] E[Y^2]$$

$$= \left(E[X^2] - \underbrace{(E[X])^2}_{=0} \right) \left(E[Y^2] - \underbrace{(E[Y])^2}_{=0} \right)$$

$$= V[X]V[Y]$$

Okay to init biases 0. Xavier: $\sigma^2 = \frac{2}{n_{in} + n_{out}}$, He: $\frac{2}{n_{in}}$ (keep activation scale constant after ReLU)

Dropout: in training, activations actually

$$a'_i = \begin{cases} 0, & \text{with prob. } p \\ \frac{a_i}{1-p}, & \text{with prob. } 1-p \end{cases}$$

Div balances expected value.

Dropout disabled during test

3 Adaptive Gradient Methods

- $f(\mathbf{x} - \eta \nabla_{\mathbf{x}} f(\mathbf{x})) = f(\mathbf{x}) - \eta (\nabla_{\mathbf{x}} f(\mathbf{x})^T \nabla_{\mathbf{x}} f(\mathbf{x})) + \mathcal{O}(\|\eta \nabla_{\mathbf{x}} f(\mathbf{x})\|^2)$
- Batch GD: $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(\theta)$, $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$. SGD: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_i(\theta)$, $E_i[\nabla_{\theta} \mathcal{L}_i(\theta)] = \frac{1}{N} \sum_{j=1}^N \nabla_{\theta} \mathcal{L}_j(\theta)$
- The best mini-batch size **B**:
 - Large enough so updates are not noisy
 - Large so computation parallelizable
 - Small, so redundancy can be exploited
- LR schedule
 - Constant (always the same): $\eta(t) = \eta$
 - Piecewise constant: $\eta(t) = \eta_i$ if $t_i \leq t \leq t_{i+1}$
 - Exponential decay: $\eta(t) = \eta_0 e^{-\lambda t}$

Adam

- Momentum responsible for direction, bad for sparse data
- RMSProp ensures rarely updated dimensions are updated more
- Momentum: $\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + (1-\beta) \mathbf{g}_t$
- RMSProp: $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + (1-\gamma) (\mathbf{g}_t \odot \mathbf{g}_t)$
- Bias corr: $\hat{\mathbf{v}}_t \leftarrow \frac{\mathbf{v}_t}{1-\beta^t}$, $\hat{\mathbf{s}}_t \leftarrow \frac{\mathbf{s}_t}{1-\gamma^t}$
 - Since β and γ start as 0, bias correction is necessary, otherwise these values will be close to 0
- Update: $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}_t + \varepsilon}} \odot \hat{\mathbf{v}}_t$
- As $t \rightarrow \infty$, $1-\beta^t \rightarrow 1$, so $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta^t} \approx \mathbf{v}_t$
- Smoothens updates and scales them, but triples memory RQs
- $\beta = 0.9$, $\gamma = 0.999$, $\eta = 0.001$, $\varepsilon = 10^{-8}$.

4 CNNs

Larger kernels capture useful structure early.

- 1ch: $H_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{(i+a),(j+b)}$
- 3ch: $H_{i,j,d} = u_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c=1}^3 V_{a,b,c,d} X_{(i+a),(j+b),c}$
- Bias scalar per channel (opt layer)**

Receptive field:

- $R_i = R_{i-1} + (K_i - 1) S_{i-1}$
 - R_i : receptive field at layer i , $R_0 = 1$
 - K_i : kernel width/height at layer i
 - S_i : cumulative stride = $\prod_{j=1}^i s_j$ where s_j is the stride of that layer, $s_0 = 1$ and $S_0 = 1$.

Output height: $OH = \left\lfloor \frac{IH + 2p - KH}{s} \right\rfloor + 1$

Padding: use odd kernel size. Output size = input size $\Leftrightarrow + \frac{k-1}{2}$

Stride: Kernels exactly don't overlap but touches full

input.

Parameter count:

- WEIGHTS = FW $\times$ FH $\times$ no. input ch $\times$ no. output ch
 - BIASES = no. output ch
 - TOTAL = WEIGHTS + BIASES
 - If must find padding, remove floor, & ceil padding
- Pooling:** Max pooling is the most common. **Always stride by pooling size, ensure no overlap. Pooling is parameterless.**

- Effects:** (1) reduce parameters & computations. (2) Extract dominant features regardless of exact positions. (3) Helps achieve invariance to small translations and distortions in input.

In classification, flatten afterwards.

4.1 BatchNorm

- Consider hidden layer w/ activations $\mathbf{x} \in \mathbb{R}^{B \times D}$
- Compute batch's mean activation:

$$\mu_i = \frac{1}{|B|} \sum_{b=1}^B x_{b,i}, i = 1 \dots D$$

- Compute batch variance:

$$\sigma_i^2 = \frac{1}{|B|} \sum_{b=1}^B (x_{b,i} - \mu_i)^2, i = 1 \dots D$$

- Perform normalization:

$$\text{BN}(x_{b,i}) = \gamma_i \odot \frac{x_{b,i} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta_i, i = 1 \dots D, b = 1 \dots B$$

- Where $\mu_{ema} \leftarrow p \mu_{ema} + (1-p) \mu$ during training and use pre-computed EMA during inference.
- Concern:** Model output per example depends on batch selected. Violates expectation that predictions are independent of batch.

4.1.1 Batch Normalization of Images

Each image has shape batch $\times$ ht $\times$ wt $\times$ channel. BatchNorm computes mean μ_B and std σ_B by averaging over batch, height, and width dimensions. Features at different locations are normalized identically. This can detect features regardless of their position.

Example: 32 RGB images, each 64×64 .

- Input shape [32, 64, 64, 3]
- For red channel: compute μ_B and σ_B across all 32 images and 64×64 positions per image.

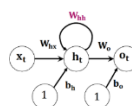
5 RNNs, Backprop

Autoregressive: An autoregressive (AR) model is a statistical and machine learning technique that predicts future values in a sequence based on its own previous values

- Sequences decomposable as $p(x_1, \dots, x_T) = p(x_1) p(x_2 | x_1) p(x_3 | x_1, x_2)$
- New samples via autoregressive sampling: $x_1 \sim p(x_1), x_2 \sim p(x_2 | x_1)$
- RNN accepts variable-length prefix, prediction depends on **all** prev. elements, fixed no. of parameters.

5.1 RNNs

One hidden layer: $\tilde{\mathbf{h}} = \sigma(\mathbf{W}_h \tilde{\mathbf{x}} + \tilde{\mathbf{b}}_h)$ and $\mathbf{o} = \mathbf{W}_o \tilde{\mathbf{h}} + \tilde{\mathbf{b}}_o$



Autoregressive prediction:

- $\mathbf{x}_1 \rightarrow \mathbf{h}_1 \rightarrow \mathbf{o}_1 \rightarrow \text{predict } \mathbf{x}_2$
- $(\mathbf{x}_2 \text{ and } \mathbf{h}_1) \rightarrow \mathbf{h}_2 \rightarrow \mathbf{o}_2 \rightarrow \text{predict } \mathbf{x}_3$
- Repeat for full sequence.

5.2 Gradients, Vectorization

Vanilla Forward pass

- PRE-ACT:** $\mathbf{z}_j^L = \sum_{i=1}^{D^{L-1}} \mathbf{W}_{ji}^L \mathbf{a}_i^{L-1} + \mathbf{b}_j^L$
 - $\mathbf{z}^L = \mathbf{W}^L \mathbf{a}^{L-1} + \mathbf{b}^L$
 - $\mathbf{Z}^L = \mathbf{A}^{L-1} (\mathbf{W}^L)^T + \mathbf{1}_N (\mathbf{b}^L)^T$
- POST-ACT:** $\mathbf{a}_j^{(1)} = \text{ReLU}(\mathbf{z}_j^{(1)})$
 - $\mathbf{a}^L = \text{ReLU}(\mathbf{z}^L)$, $\mathbf{A}^L = \text{ReLU}(\mathbf{Z}^L)$

$$\text{SOFTMAX: } \mathbf{a}_k^{(2)} = \frac{e^{\mathbf{z}_k^{(2)}}}{\sum_{k'=1}^{D^{(2)}} e^{\mathbf{z}_{k'}^{(2)}}}$$

$$\mathbf{a}^{(2)} = \frac{\exp(\mathbf{z}^{(2)})}{\mathbf{1}_{D^{(2)}} \exp(\mathbf{z}^{(2)})}$$

$$\mathbf{A}^{(2)} = \frac{\exp(\mathbf{Z}^{(2)})}{\exp(\mathbf{Z}^{(2)}) \mathbf{1}_{D^{(2)}}}$$

Vanilla Backward pass

- $\mathbf{z}^L = \mathbf{W}^L \mathbf{a}^{L-1} + \mathbf{b} \approx \mathbf{A}^{L-1} (\mathbf{W}^L)^T + \mathbf{1}_N (\mathbf{b}^L)^T$
- $\nabla_{\mathbf{W}^L} \mathcal{L} = (\nabla_{\mathbf{z}^L} \mathcal{L}) (\mathbf{a}^{L-1})^T \approx (\nabla_{\mathbf{z}^L} \mathcal{L})^T \mathbf{A}^{L-1}$
- $\nabla_{\mathbf{a}^{L-1}} \mathcal{L} = (\mathbf{W}^L)^T (\nabla_{\mathbf{z}^L} \mathcal{L}) = (\nabla_{\mathbf{z}^L} \mathcal{L}) \mathbf{W}^L$
- $\nabla_{\mathbf{z}^{L-1}} \mathcal{L} = (\nabla_{\mathbf{a}^{L-1}} \mathcal{L}) \odot \sigma'(\mathbf{z}^{L-1})$

RNN Forward pass

- $\tilde{\mathbf{z}}_t = \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{b}_h$
- $\tilde{\mathbf{h}}_t = \sigma(\tilde{\mathbf{z}}_t)$

- $\vec{o}_t = \mathbf{W}_o \vec{h}_t + \vec{b}_o$
- $L = \frac{1}{T} \sum_{t=1}^T L_t(\vec{x}_t, \vec{o}_t)$

RNN Backward pass

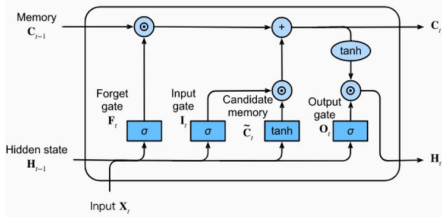
- $\nabla_{\mathbf{w}_o} L = \sum_{t=1}^T (\nabla_{\mathbf{o}_t} L) \cdot (\mathbf{h}_t)^T$
- $\nabla_{\mathbf{w}_{hx}} L = \sum_{t=1}^T (\nabla_{\mathbf{z}_t} L) \cdot (\mathbf{x}_t)^T$
- $\nabla_{\mathbf{w}_{hh}} L = \sum_{t=1}^T (\nabla_{\mathbf{z}_t} L) \cdot (\mathbf{h}_{t-1})^T$
- $\nabla_{\mathbf{z}_t} L = \nabla_{\vec{h}_t} L \odot \sigma'(\mathbf{z}_t)$
- $\nabla_{\mathbf{h}_t} L = \mathbf{W}_o^T (\nabla_{\mathbf{o}_t} L) + \mathbf{w}_{hh}^T (\nabla_{\mathbf{z}_{t+1}} L)$

Product rule

- $\frac{d}{dx} f(x)g(x) = f'(x)g(x) + f(x)g'(x)$

5.3 LSTM

- **Forget gate:** $\mathbf{F}_t = \sigma(\mathbf{W}_{if} \mathbf{x}_t + \mathbf{W}_{hf} \mathbf{h}_{t-1} + \mathbf{b}_f)$
- **Input gate:** $\mathbf{I}_t = \sigma(\mathbf{W}_{xi} \mathbf{x}_t + \mathbf{W}_{hi} \mathbf{h}_{t-1} + \mathbf{b}_i)$
- **Output gate:** $\mathbf{O}_t = \sigma(\mathbf{W}_{xo} \mathbf{x}_t + \mathbf{W}_{ho} \mathbf{h}_{t-1} + \mathbf{b}_o)$
- **Candidate value:** $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_{xc} \mathbf{x}_t + \mathbf{W}_{hc} \mathbf{h}_{t-1} + \mathbf{b}_c)$



No repeated matrix multiplications or non-linearity.

- Cell state: $\mathbf{c}_t = \mathbf{F}_t \odot \mathbf{c}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{c}}_t$
- Gradient flow: $\frac{\partial L}{\partial \mathbf{c}_t} = \frac{\partial L}{\partial \mathbf{c}_{t+1}} \odot \mathbf{F}_{t+1}$

For **encoder-decoder**: Encoder – spit out hidden state.
Decoder: init hidden state w/ enc hidden state

6 Attention

Var	Dim	Comp
$\mathbf{X}$	$T \times d_{\text{model}}$	input
$\mathbf{W}_Q$	$d_k \times d_{\text{model}}$	learnt
$\mathbf{W}_K$	$d_k \times d_{\text{model}}$	learnt
$\mathbf{W}_V$	$d_v \times d_{\text{model}}$	learnt
$\mathbf{Q}$	$T \times d_k$	$\mathbf{XW}_Q^T$
$\mathbf{K}$	$T \times d_k$	$\mathbf{XW}_K^T$
$\mathbf{V}$	$T \times d_v$	$\mathbf{XW}_V^T$
$\mathbf{S}$	$T \times T$	$\mathbf{QK}$
$\mathbf{A}$	$T \times T$	$\text{softmax}\left(\frac{\mathbf{S}}{\sqrt{d_k}}\right)$
$\mathbf{H}$	$T \times d_v$	$\mathbf{AV}$
$\mathbf{H}^{(i)}$	$T \times \frac{d_{\text{model}}}{H}$	$\mathbf{Q}^{(i)}, \mathbf{K}^{(i)}, \mathbf{V}^{(i)}$: are $T \times \frac{d_{\text{model}}}{H}$
$\mathbf{W}_O$	$d_{\text{model}} \times d_{\text{model}}$	learnt
$\mathbf{O}$	$T \times d_{\text{model}}$	$\text{concat}(\mathbf{H}^{(1)} \dots \mathbf{H}^{(H)}) \mathbf{W}_O$

In **MHSA**: $d_k = d_v = \frac{d_{\text{model}}}{H}$. $\mathbf{W}_O$ mixes information across heads and restores the original dimension. **Causal masking applies to S before A is computed.**

- The comparison step computes $\mathbf{S}$
 - Where queries are compared against keys: $\alpha(\mathbf{q}_i, \mathbf{k}_j) \in \mathbb{R}$
- The reduction step computes $\mathbf{H}$
 - This computes a weighted average of values from $\mathbf{V}$. $f(\{\mathbf{v}_1, \dots, \mathbf{v}_N\}, \mathbf{a})$
- Matrix-less form for seq2seq:
 - $\alpha(\mathbf{q}_i, \mathbf{k}_j) = \mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}_j) = a_i$
 - $\mathbf{c}_t = \sum_{j=1}^T \text{softmax}(\mathbf{a}_i) \mathbf{h}_j$
 - $\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}_t)$

6.1 Where the values come from

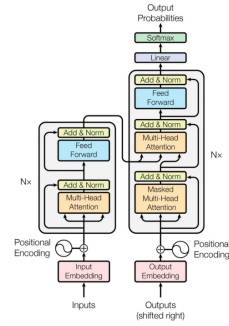
- Q
 - 1NN: data point to be classified
 - Seq2seq: Prev decoder hidden state, $\mathbf{q} = \mathbf{s}_{t-1}$
- K
 - 1NN: all training data points
 - Seq2seq: Encoder hidden states
- V
 - 1NN: each training point label
 - Seq2seq: Encoder hidden states

6.2 Enc Decoder

An encoder processes the source sequence and produces a context vector for the decoder to start with. The decoder starts with <BOS> as the current produced state.

Cross attention

Encoder outputs are mixed with cross-attention step in the decoder



6.3 Pos enc

- Relative key: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = \mathbf{q}_i^T (\mathbf{k}_j + \mathbf{p}_{i-j}) / \sqrt{d_k}$, $\mathbf{p}_{i-j} \in \mathbb{R}^{d_k}$ learnt
- Rel attn bias: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = \mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_k} + p_{i-j}$, $p_{i-j} \in \mathbb{R}$ learnt
- Rotary pos enc: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = (\mathbf{R}_i \mathbf{q}_i)^T (\mathbf{R}_j \mathbf{k}_j) / \sqrt{d_k}$, $\mathbf{R}_i$ hardcoded per pos

7 Transformers

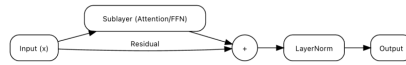
Layer norm: (per feat) For all $\mathbf{h}_t \in \mathbb{R}^d$, $t \in \{1 \dots T\}$:

- $\mu_t = \frac{1}{d} \sum_{j=1}^d h_{t,j}$, $\sigma_t^2 = \frac{1}{d} \sum_{j=1}^d (h_{t,j} - \mu_t)^2$
- $\text{LN}(\mathbf{h}_t)_i = \gamma_i \frac{h_{t,i} - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}} + \beta_i$

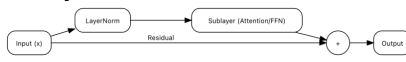
Pos-wise FFN: $\mathbf{h}_t = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}_t + \mathbf{b}_1) + \mathbf{b}_2$.

$\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ffn}} \times d}$, $d_{\text{ffn}} = 4d$

Post-layer norm (note: attn/FFN is EITHER not both)



Pre-layer norm



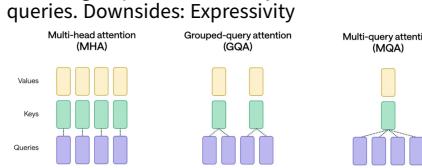
SwiGLU

- $\text{FFN}(\mathbf{x}_t) = \mathbf{W}_3 ((\mathbf{W}_1 \mathbf{x}_t) \odot \text{Swish}(\mathbf{W}_2 \mathbf{x}_t))$
- $\text{Swish}(\mathbf{x}_t) = \mathbf{x}_t \odot \sigma(\mathbf{x}_t)$
 - Two branches: content and gate
 - Gate controls signal element wise
 - Improve expressiveness
 - FFN with Swi GLU: swish is smooth, nonmonotonic, improve grad flow; intermediate dim reduced to 2/3 of original to keep parameter count similar

Variants

- **Flash attn:** keep data in fast GPU mem, softmax computed on fly, faster
 - Do not store the full $T \times T$ attention matrix.
 - Uses much less memory than standard attention.
 - Runs much faster in practice.
 - Produces the exact same result as standard attention.
- **Local:** QKV window size limited w/ size W
 - May increase path length – long-range interactions require multiple layers. **Fix:** alternate local & global attention layers

GCA – G groups. Most diversity of heads come from queries. Downsides: Expressivity



Encoder only

- Repeats the transformer block L times. Each block takes an input sequence of length T and outputs a sequence of length T .
- Seq label: apply classification head, pred 1 label per pos
- Seq class: reserve start for label
- Span extr: concat question and passage; predict start & end score per token

Decoder only

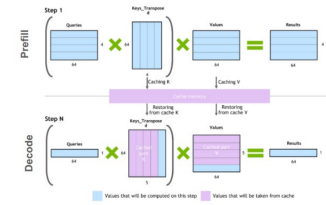
- Used for autoregressive sequence generation
 - Back to original language modeling
 - Given past tokens, predict next
- Generates: [BOS], “the”, “cat”, “sat”
- When generating “cat”, can only attend to [BOS] and “the” (and itself)
- Problems with dumber approaches:
 - Sequential training: correct but slow
 - Naive parallel: model can cheat by attending to future tokens

- Causal mask: $\mathbf{M} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}, S + M$

K/V caching

- Keys and values for past tokens do not change during generation, as:
 - Weights are fixed during inference
 - Token representation does not depend on future tokens
- Idea: cache keys (**K**) and values (**V**) from prev steps and reuse them.
- Trade-off: cache size grows with sequence length

$(Q * K^T) * V$ computation process with caching



Pre-training

- Done b/c inductive bias for transformers weak.
- **Autoregressive:** Model each token based on previous
- **MLM:** Make a hole, predict the hole
- **BERT:** Make 15% of holes, predict them

Enc-Decoder / Cross attention

- Cross attn: **enc** provides KV as output
- In cross attn – Q: dec states, KV: enc outputs, Comparison: scaled dot prod, Reduction: softmax-wt avg
- To train Enc-Dec transformers: use seq2seq, encoder processes input, decoder autoregressively generates target. For pre-training, use span corruption (mask random words)
- See previous section for diagram

Bias removal

Remove bias from equations: $\mathbf{h}_t = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}_t)$.

$\text{LN}(\mathbf{h}_t)_i = \gamma_i \frac{h_{t,i}}{\sqrt{\frac{1}{d} \sum_{j=1}^d (h_{t,j})^2 + \epsilon}}$. Done because it doesn't hurt performance.

8 LLMs

Training data is just text: No annotation needed, no sentiment labels, no topic tags, no translations. Massive scale: from 10M examples to 10T tokens. More Parameters + More Training = Lower Loss. Lower Loss $\Rightarrow$ better at all text-based tasks:

- Question answering, Paraphrase detection, Summarization, Translation

Prompting matters: I'm sorry, I can't complete this request. Solutions:

- Better rephrased prompt (This is...)
 - Probability inspection
 - Check the model's internal probabilities for specific tokens (e.g. yes/no)
 - Few-shot in-context learning
 - Pros: Improves performance, no param updates, same model for all tasks.
 - Cons: Cost grows with examples, need larger models, worse than fine-tuning
- Post-training: from base models to instruction-following agents
- Instruction tuning: train on diverse query-response pairs. Leads to better generalization
 - Train on good vs. bad outputs: make more likely, make less likely

9 Vision Transformers

CNNs have strong inductive biases, transformers don't. (Self-attention requires every output element to depend on every input element, and makes fewer structural assumptions about the data)
Transformers are flexible and can learn useful dependencies from data. Attention can learn to focus on local regions.

9.1 Image to Sequence

Training and scaling behavior

- Take $P \times P$ patch from image, including all channels and flatten
- Treat that as one element of the input sequence
- For image with $H \times W \times C$, input seq len = $M = \frac{HW}{P^2}$. Each element has dim $P^2 C$
- Classification: map img to len M seq using patches, prepend [CLS] token, linearly project elem to dim d_{model} , process through enc-only stack, feed rep for [CLS] into classifier.
- CNNs better w/ small datasets, ViTs better on large. Weaker inductive biases need more data but yields more flexible representations.

10 LoRA

Fine-tuned models represented by: $\mathbf{W}_{\text{FT}} = \mathbf{W}_{\text{PRE}} + \Delta\mathbf{W}$ where $\Delta\mathbf{W}$ represents all updates via gradient descent. LoRA approximates $\Delta\mathbf{W}$ as a low-rank matrix: $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$ where $\mathbf{B} : (N_{\text{out}} \times R)$, $\mathbf{A} : (R \times N_{\text{in}})$; $R \ll N_{\text{out}}, N_{\text{in}}$. R is the rank, usually 1, 8, 16.

Small R works b/c: pre-training learns broad, fine-tuning targets narrow, only need to steer, not rewrite.

Training: Given $\mathbf{W}_{\text{PRE}} : (N_{\text{out}} \times N_{\text{in}})$, add new params $\mathbf{B}\mathbf{A}$ for $\mathbf{W}_{\text{FT}} = \mathbf{W}_{\text{PRE}} + \mathbf{B}\mathbf{A}$; during training, freeze $\mathbf{W}_{\text{PRE}}$ and train only $\mathbf{A}$ and $\mathbf{B}$.

Initialization: To avoid corrupting pre-trained model at start, $\mathbf{B} = \mathbf{0}$, $\mathbf{A} \sim \mathcal{N}(\mathbf{0}, \sigma^2)$. At start, $\mathbf{B}\mathbf{A} = \mathbf{0}$, same as pre-trained.

LoRA does not need to be applied to every layer and stores only 1-10% of full model (update only attn or MLP or both). Cheaper to store. No extra inference cost.

11 Mixture of Experts

MoE decouples parameter count from compute cost. Have n experts instead of one layer. Each expert has same architecture but different params.

Adding this many experts will result in n additional parameters, but compute only increases by k since we're computing k experts at a time, $k \ll n$. You get the benefits of a large model while paying the computational cost for a fraction of the network.

A router decides which k experts are active and process the input. $\mathbf{p} = \text{softmax}(\mathbf{W}_r \mathbf{x})$; $\mathbf{p}$ is the probabilities over experts. Select the top k experts, $k = 1, 2$, which only process the input during training and inference.

Top k is binary; an expert only receives gradient update if it's chosen. Expert outputs are weighted by the routing probabilities: $\mathbf{y} = \sum_{e \in \text{Top } k} p_e f_e(\mathbf{x})$, $p_e = \text{softmax}(\mathbf{W}_r \mathbf{x})_e$

11.1 Router Collapse Problem

Router is trained w/o supervision. If expert i was selected more often, router collapses to a single expert.

Add reg term:

$$L_{\text{bal}} = \sum_{\text{experts}} (\text{fraction chosen}) \times (\text{router probability})$$

- Fraction chosen how often the expert is actually chosen
- Router probability: how much the router prefers the expert
- Choose all evenly: $L_{\text{bal}} = \frac{1}{n} \sum (\frac{1}{n}) = \frac{1}{n}$.
- Collapse to one: $0 \cdot 0 + \dots + 1 \cdot 1 = 1$

12 GNN

Graphs have N nodes; node feat $\mathbf{X} : (N \times D)$. Adjacency matrix: $\mathbf{A} : (N \times N)$. $A_{ij} = 1$ if node i connected to j , else 0. Can represent directed graphs; this matrix is very sparse.

12.1 Aggregating Neighbor Information

AX. Where row i is the sum of feature vectors of node i 's neighbors. Each node's representation contains info about its neighbors. $(\mathbf{AX})_i = \sum_{j=1}^N A_{ij} \mathbf{x}_j = \sum_{j \in \text{neighbor}(i)} \mathbf{x}_j$.

Nodes w/ more neighbors produce larger sums since no normalization by node degrees.

12.2 Normalized Neighbor Aggregation

$\mathbf{D} : (N \times N)$, is diagonal. $d_i = \sum_{j=1}^N A_{ij}$ = degree (neighbor count) of node i .

Normalized neighbor aggregation:

$$\sum_{j=1}^N \frac{A_{ij}}{\sqrt{d_i d_j}} \mathbf{x}_j = (\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{X})_i$$

Every neighbor feature vector is normalized using both endpoints: d_i (deg of receiving - current node), d_j (deg of sending node - neighbor). Normalization & using the square root is symmetric and balanced; SQRT does not shrink the contribution too aggressively.

12.3 Graph Convolutional Networks

Standard: $\mathbf{H} = \phi(\mathbf{X}\mathbf{W})$

GCN layer: $\mathbf{H} = \phi(\mathbf{\hat{D}}^{-\frac{1}{2}} \mathbf{\hat{A}} \mathbf{\hat{D}}^{-\frac{1}{2}} \mathbf{X}\mathbf{W})$, where $\mathbf{X}$ is feature

matrix, $\mathbf{\hat{A}}$ is adj matrix, $\mathbf{\hat{D}}$ is deg matrix of $\mathbf{\hat{A}}$, and

$\mathbf{W} \in \mathbb{R}^{D \times D'}$ is learned projection.

$\mathbf{A}$ does not have self-connections by default - needed since we want to keep the nodes' own features. Hence:

$\mathbf{\hat{A}} = \mathbf{A} + \mathbf{I}$; each node treated as its own neighbor.

Called a graph convolution since GCN aggregates info from its neighbors, GCN has the same wt matrix applied to every node, and GCN transforms neighbor features with $\mathbf{W}$ and sum them. Stacking GCN layers expands receptive fields; more layers $\Rightarrow$ info from farther nodes in the graph.

12.3.1 Node Classifications

After stacking GCN layers:

- Each node i has a feat vector $\mathbf{h}_i$
 - Which contains info about itself, its neighbors, its neighbors of neighbors
- Predict the label of node i using a linear classifier:

$$\mathbf{Y} = \text{softmax}(\mathbf{H}\mathbf{W}_c), \mathbf{H} \in \mathbb{R}^{N \times D}, \mathbf{W}_c \in \mathbb{R}^{D \times C}$$

12.4 Graph Attention Networks (GAT)

GCNs have all neighbors contribute equally. Attention mechanisms can be used to have it weigh some neighbors more.

Each node has feature vector $\mathbf{h}_i$, $Q = K = V = \mathbf{W}\mathbf{h}_i$,

$a_{ij} = \text{LeakyReLU}(\mathbf{v}^T (\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j))$ where $\mathbf{v}$ is learned and $\parallel$ is learned.

Reduction: $\mathbf{h}'_i = \sigma(\sum_{j \in \text{neighbors}(i)} \text{softmax}(\mathbf{a}_{ij}) \mathbf{W}\mathbf{h}_j)$.

Implement sum over neighbors via masking using adjacency matrix. Uses MHA.

Completing: Stack - GAT $\rightarrow$ GAT $\rightarrow$ GAT. Add classification head: $\mathbf{Y} = \text{softmax}(\mathbf{H}\mathbf{W}_c)$. Optional: insert pos-wise MLP - $\mathbf{h}_i \rightarrow \text{MLP}(\mathbf{h}_i)$

13 VAE

13.1 Representation Learning

In trad ML, design features by hand. In DL, NNs learn features automatically. Learning features $\rightarrow$ learned representations from data. Goal: learn useful representation from unlabeled data for downstream tasks.

Given an input, representation has as little info as possible about the input. The input can still be reconstructed from the representation.

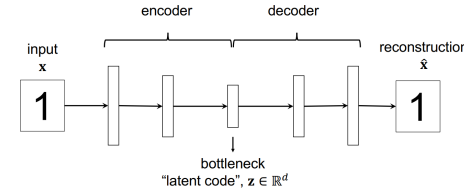
Store informative features instead of pixels of an image. For digits: digit identity, stroke thickness, slant, etc. Learn a compact representation that preserves essential image structure.

Why?

- Compression forces model to capture salient features
- These make downstream tasks easier than raw data
- Compression and reconstruction enable learning without labels

13.2 Autoencoder Architecture

The purpose of an autoencoder - data compressor:



Train using reconstruction loss: $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. If $\dim(\mathbf{z}) < \dim(\mathbf{x})$, no, compression is not guaranteed because $\mathbb{R}$ has infinite values. If limited to discrete, gradients cannot flow. Instead, we use VAE to limit information in the latent code.

VAE as a probabilistic generative model:

- Sample latent code: $\mathbf{z} \sim p(\mathbf{z})$
- Generate data: $\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$
- $\mathbf{z}$ encodes salient factors of the data
- Generate an image as follows:
 - First, sample values for salient factors, then use them to produce the image
- MLE is intractable: $\max_{\theta} (\log p_{\theta}(\mathbf{x}))$ where $p_{\theta}(\mathbf{x})$ is the probability of data point $\mathbf{x}$.
 - Given the latent variable model with $p_{\theta}(\mathbf{x}|\mathbf{z})$ and $p(\mathbf{z})$, $p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$; integrating over all values of latent $\mathbf{z}$ is intractable.
 - Variational inference: maximize tractable lower bound on $\log p_{\theta}(\mathbf{x})$

13.3 Evidence Lower Bound (ELBO)

- Start with likelihood of data: $\log p_{\theta}(\mathbf{x}) = \log \int p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z}$
- Introduce approximate posterior $q_{\phi}(\mathbf{z}|\mathbf{x})$:
 - $\log p_{\theta}(\mathbf{x}) = \log \int q_{\phi}(\mathbf{z}|\mathbf{x}) \frac{p_{\theta}(\mathbf{x}|\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} d\mathbf{z}$
- Rewrite integral as an expectation:
 - $\log p_{\theta}(\mathbf{x}) = \log \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\theta}(\mathbf{x}|\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right]$
- Apply Jensen's inequality: $f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$ for concave f
 - $\log p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right]$
- Since $p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z})$:
 - $\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] = \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x}) p(\mathbf{z})} \right]$
 - $= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right]$
- Define $D_{\text{KL}}(Q \| P) = \mathbb{E}_Q \left[\log \frac{Q}{P} \right]$:
 - $\Rightarrow \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$

VAE objective notation:

- $\mathbf{z}$: latent code, RV
- $p(\mathbf{z})$: prior over latent codes, usually a standard normal
- $p_{\theta}(\mathbf{x}|\mathbf{z})$: likelihood - dist over data points $\mathbf{x}$ given

latent code $\mathbf{z}$

- $q_{\phi}(\mathbf{z}|\mathbf{x})$: approximate posterior, distribution over latent codes $\mathbf{z}$ given input $\mathbf{x}$

Objective:

$$\max_{\theta, \phi} \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$$

Left term: Reconstruction term - reconstruct the input accurately. Maximize likelihood of input, given latent code.

- If $p_{\theta}(\mathbf{x}|\mathbf{z})$ is modeled as a Gaussian:

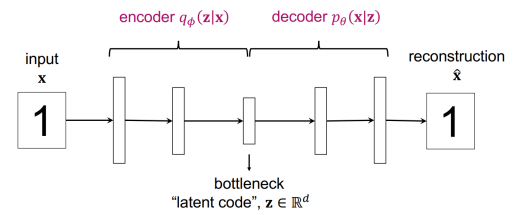
$$\log p_{\theta}(\mathbf{x}|\mathbf{z}) \propto -\|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

- This term becomes the mean squared reconstruction error.

Right term: Regularization term - keep the latent code simple. Minimize distance from latent distribution to prior.

- Measures distance between latent dist $q_{\phi}(\mathbf{z}|\mathbf{x})$ and prior $p(\mathbf{z})$.
- Typically, $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$

13.4 VAE as NN



Encoder network models approximate posterior.

Encoder predicts which latent codes explain the input. Encoder takes $\mathbf{x}$ and outputs parameters of latent dist $q_{\phi}(\mathbf{z}|\mathbf{x})$, where $q_{\phi}(\mathbf{z}|\mathbf{x})$ is typically a Gaussian w/ diag covariance for computational eff:

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mu_{\phi}(\mathbf{x}), \text{diag}(\sigma_{\phi}^2(\mathbf{x}))).$$

Encoder receives input $\mathbf{x}$ and outputs $\mu_{\phi}(\mathbf{x})$, $\sigma_{\phi}^2(\mathbf{x})$. If $\mathbf{z} \in \mathbb{R}^d$, enc output size = $2d$ (d means and d variances).

Means are unconstrained but variances must be positive (softplus non-linearity).

Decoder network models likelihood

Decoder predicts the data point $\mathbf{x}$ generated by latent code $\mathbf{z}$. Decoder receives latent code $\mathbf{z}$ as input and outputs params of dist $p_{\theta}(\mathbf{x}|\mathbf{z})$. Example: decoder with Gaussian likelihood: $p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mu_{\theta}(\mathbf{z}), \mathbf{I})$. Choice of dist depends on whether data is continuous or discrete. The KL divergence has a closed-form expr:

$$D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

This term encourages $\mu_j \rightarrow 0$ and $\sigma_j^2 \rightarrow 1$. The KL term

pushes $q_{\phi}(\mathbf{z}|\mathbf{x})$ to stay as close to the prior $p(\mathbf{z})$.

Challenges with Training a VAE

- Feed input $\mathbf{x}$ into encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$
- Sample latent code $\mathbf{z}$ from $q_{\phi}(\mathbf{z}|\mathbf{x})$
- Feed latent code $\mathbf{z}$ into decoder $p_{\theta}(\mathbf{x}|\mathbf{z})$
- Compute loss and backpropagate

Problem: Sampling is stochastic and not differentiable. Solution: the reparameterization trick:

$\mathbf{z} = \mu(\mathbf{x}) + \sigma(\mathbf{x}) \odot \epsilon$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (fixed), which makes $\mathbf{z}$ a deterministic, differentiable function of μ , σ . This is mathematically equivalent to sampling $\mathbf{z}$ directly from $\mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x}))$.

14 Transposed Convolution

To make a VAE for images, use convolutions. Encoder can shrink spatial dims using convolution + stride and pooling. So, how do we map the latent code back to the original image size - use transposed convolution. It's a convolution like operation to increase spatial dims, allowing decoder to learn how to up-sample feature maps.

A convolution can be written as a matrix-vector product: $\mathbf{C} \cdot \text{vec}(\mathbf{X}) = \text{vec}(\mathbf{Y})$. Flatten $\mathbf{X}$ into a vec, each row of $\mathbf{C}$ applies kernel at one location, produces output $\mathbf{Y}$ as a vector. **This is still slightly lossy.**

Transposed convolution transposes $\mathbf{C}$. This naturally appears in backprop, where the gradient of convolution is the transposed convolution:

- Conv: $\text{vec}(\mathbf{Y}) = \mathbf{C} \cdot \text{vec}(\mathbf{X})$
- Transposed conv: $\text{vec}(\frac{\partial \mathcal{L}}{\partial \mathbf{X}}) = \mathbf{C}^T \cdot \text{vec}(\frac{\partial \mathcal{L}}{\partial \mathbf{Y}})$

Transpose flips input/output roles, but is still a matrix-vector product

Setup: $\mathbf{V}$ is a 2×2 feature map, $\mathbf{K}$ is a 2×2 kernel, stride 2. $\mathbf{U}$ is a 4×4 output: $\mathbf{C}^T \cdot \text{vec}(\mathbf{V}) = \text{vec}(\mathbf{U})$, where $\mathbf{C}$ is the convolutional matrix and $\text{vec}(\cdot)$ is the flattening operation.

- Each column of $\mathbf{C}$ becomes a row of $\mathbf{C}^T$
- Each input in $\mathbf{V}$ stamps a scaled kernel onto the output

U

- Overlapping contributions are summed

14.1 Visualizing Transposed Convolution

Each input value stamps a scaled kernel onto the output

For each input value Z_{ij} :

- Multiply kernel by Z_{ij}
- Paste it into the output at post (i,j)
- Shift determined by stride
- Repeat for all input entries
- If pasted regions overlap, add the values

$$\mathbf{V} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{K} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{U} = \begin{bmatrix} 1 & 2 & 2 & 4 \\ 3 & 4 & 6 & 8 \\ 3 & 6 & 4 & 8 \\ 9 & 12 & 12 & 16 \end{bmatrix}$$

Flatten $\mathbf{U}$, each row of ending matrix $\mathbf{C}$ corresponds to one output entry such as $\text{vec}(\mathbf{U})$. Note that $\mathbf{C}$'s dimensions are $\text{OUTPUT ELEMENTS} \times \text{INPUT ELEMENTS}$. Notice that the input elements were flattened.

15 U-Net

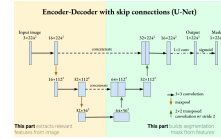
- IM $\rightarrow$ IM ex: semantic segmentation, a class per pixel, opt dims must match input
- Encoder use convolutions and pooling to produce smaller feature maps.
- Decoder use transposed convolutions to upsample back to input resolution.
- Skip connections via concatenation to reintroduce local features.

For Fully convolutional networks, there are no fully connected layers since fully connected layers destroy

spatial structure and require fixed input size.

However, the encoder-decoder:

- Excels at extracting global features
- How do we remember local features?



Use skip connections. Deep conv nets can forget local features as it passes info through layers. Skip connections can reintroduce such information to improve learning. Decoder incorporates feature map from the corresponding encoder at each step.